Report 5: Beam Search Aggregation & N-Best Hypothesis Fusion

Document Classification: Technical Research Report **Date:** February 17, 2026 **Scope:** Leveraging all beam search candidates to improve VSP-LLM output quality **Current State:** Beam width 20, but only best-1 hypothesis is saved — 19 alternatives discarded

---

1. Executive Summary

The VSP-LLM model performs beam search with **20 candidates** at every generation step. This means the model internally produces 20 distinct hypothesis sentences, each with an associated probability score. Currently, only the single best hypothesis is saved — the other 19 are permanently discarded.

This is an extraordinary waste of information. The 20 beam candidates collectively contain far more information than any single one. Research in ASR and machine translation has shown that aggregating N-best hypotheses can reduce WER by **5-20% relative** (Fiscus, 1997; Xu et al., 2011; Sim et al., 2007).

This report analyzes 5 aggregation strategies, provides exact code modifications, and estimates the expected improvement for each.

---

2. Current State: What the Model Produces and What We Keep

2.1 Beam Search Internals

During generation, the model maintains 20 candidate sequences at each step:

```
Step 0: 20 candidates start with different first tokens
Step 1: Each candidate extends → 20 × vocab_size possibilities → keep best 20
Step 2: ...
Step N: Final 20 complete hypotheses, ranked by log-probability
```

At the end, we have:

```
Beam 0  (best):   "the speaker is discussing anatomy"         score: -12.3
Beam 1  (2nd):    "the speaker is discussing autonomy"        score: -12.8
```

```
Beam 2  (3rd):    "the speaker is discussing astronomy"       score: -13.1
Beam 3  (4th):    "the speaker discussing anatomy"            score: -13.5
...
Beam 19 (worst):  "a speaker discussing an atom"              score: -18.2
```

Currently saved: Beam 0 only **Currently discarded:** Beams 1-19 + all scores

2.2 The Information Loss

For each segment, we lose: - **19 alternative hypotheses** that may contain correct words missing from the best hypothesis - **20 sequence-level scores** that collectively indicate confidence - **Cross-beam agreement patterns** that signal reliable vs. unreliable words - **Diversity information** that reveals how "certain" the model is overall

---

3. Code Modifications to Capture N-Best

3.1 Modify `vsp_llm.py` generate()

```
# File: VSP-LLM/src/vsp_llm.py
# In the generate() method, add num_return_sequences parameter:

@torch.no_grad()
def generate(self,
             num_beams=20,
             max_length=30,
             num_return_sequences=1,     # NEW: how many hypotheses to return
             # ... other params
             **kwargs):
    # ... existing encoder + dedup code ...

    self.decoder.config.use_cache = True
    outputs = self.decoder.generate(
        inputs_embeds=llm_input,
        num_beams=num_beams,
        num_return_sequences=num_return_sequences,  # NEW
        max_new_tokens=max_length,
        # ... other params ...
        output_scores=True,                          # NEW
        return_dict_in_generate=True,                # NEW
    )

    return outputs
```

3.2 Modify `vsp_llm_decode.py`

```
# File: VSP-LLM/src/vsp_llm_decode.py
# Replace the generation and decoding section:

N_BEST = 5  # Number of hypotheses to save (configurable)

gen_output = model.generate(
    target_list=sample["target"],
    num_beams=cfg.generation.beam,
    num_return_sequences=N_BEST,       # NEW: return top-5
    max_length=dynamic_max_len,
    # ... other params ...
    **sample["net_input"]
)

# gen_output.sequences shape: [batch_size * N_BEST, seq_len]
# gen_output.sequences_scores shape: [batch_size * N_BEST]

all_hypos = tokenizer.batch_decode(
    gen_output.sequences,
    skip_special_tokens=True,
    clean_up_tokenization_spaces=False
)

# Reshape: group by segment
for i in range(len(sample["id"])):
    segment_hypos = all_hypos[i * N_BEST : (i + 1) * N_BEST]
    segment_scores = gen_output.sequences_scores[i * N_BEST : (i + 1) * N_BEST].tolist()

    result_dict['utt_id'].append(sample['utt_id'][i])
    result_dict['hypo'].append(segment_hypos[0])  # Best hypothesis (backward compat)
    result_dict['hypo_nbest'].append(segment_hypos)  # NEW: all N hypotheses
    result_dict['scores_nbest'].append(segment_scores)  # NEW: all N scores

    # Apply aggregation strategy
    aggregated = aggregate_nbest(segment_hypos, segment_scores)
    result_dict['hypo_aggregated'].append(aggregated)  # NEW: aggregated result
```

3.3 Memory Considerations

Returning N hypotheses from beam search: - **N=5, beam=20:** Each segment returns 5 (not 20) sequences. The beam search still explores 20 candidates internally, but only materializes the top 5 for output. Memory overhead: ~5x the current single-hypothesis storage. Manageable. - **N=20, beam=20:** Returns all 20 candidates. Higher information content but 20x storage and decode time. Only recommended for research/analysis.

Recommendation: Use N=5 for production, N=20 for analysis runs.

---

4. Aggregation Strategies

4.1 Strategy 1: Minimum Bayes Risk (MBR) Decoding

Expected improvement: 5-10% relative WER reduction **Complexity:** Medium

Concept

Instead of selecting the hypothesis with the highest probability (MAP decoding = current approach), select the hypothesis that **minimizes the expected error** across all candidates. MBR selects the hypothesis that is most "central" — the one most similar to all others.

Algorithm

```
import editdistance

def mbr_decode(hypotheses, scores):
    """Select the hypothesis that minimizes expected WER across all candidates.

    Args:
        hypotheses: List of N hypothesis strings
        scores: List of N log-probability scores

    Returns:
        Best hypothesis according to MBR criterion
    """
    n = len(hypotheses)

    # Convert log-scores to probabilities (normalized)
    import math
    max_score = max(scores)
    probs = [math.exp(s - max_score) for s in scores]
    total = sum(probs)
    probs = [p / total for p in probs]

    # For each candidate, compute expected WER against all others
    expected_wer = []
    for i in range(n):
        hyp_i_words = hypotheses[i].split()
        weighted_wer = 0.0
        for j in range(n):
            hyp_j_words = hypotheses[j].split()
            dist = editdistance.eval(hyp_i_words, hyp_j_words)
            wer = dist / max(len(hyp_j_words), 1)
            weighted_wer += probs[j] * wer
        expected_wer.append(weighted_wer)

    # Select hypothesis with minimum expected WER
```

```
    best_idx = expected_wer.index(min(expected_wer))
    return hypotheses[best_idx]
```

Why This Helps

The MAP hypothesis (highest probability) may be an outlier — a high-probability but unusual sequence. MBR selects the "consensus" hypothesis that best represents the distribution. For lip reading, where visual features are ambiguous, the consensus is often more accurate than the single best guess.

Literature

- Kumar & Byrne (2004). "Minimum Bayes-Risk Decoding for Statistical Machine Translation." HLT-NAACL 2004.
- Eikema & Aziz (2022). "Sampling-Based Approximations to Minimum Bayes Risk Decoding." EMNLP 2022.
- Freitag et al. (2022). "High Quality Rather Than High Model Probability: MBR Decoding with Neural Metrics." TACL 2022.

4.2 Strategy 2: ROVER (Recognizer Output Voting Error Reduction)

Expected improvement: 5-15% relative WER reduction **Complexity:** Medium-High

Concept

ROVER is the gold standard for combining multiple ASR hypotheses. It works at the **word level** rather than the sequence level:

1. **Alignment:** Align all N hypotheses word-by-word using dynamic programming
2. **Voting:** At each word position, vote across hypotheses (weighted by score)
3. **Selection:** Choose the most-voted word at each position

Algorithm

```
def rover_combine(hypotheses, scores, null_penalty=0.5):
    """ROVER-style word-level voting across N-best hypotheses.

    Args:
        hypotheses: List of N hypothesis strings
        scores: List of N log-probability scores (for weighting)
        null_penalty: Penalty for voting to delete a word position

    Returns:
        Combined hypothesis string
    """
    import math
```

```
    # Convert scores to weights
    max_score = max(scores)
    weights = [math.exp(s - max_score) for s in scores]
    total = sum(weights)
    weights = [w / total for w in weights]

    # Step 1: Build word lists
    word_lists = [h.split() for h in hypotheses]

    # Step 2: Pairwise alignment using edit distance alignment
    # Start with first hypothesis as backbone
    backbone = word_lists[0]

    # Simplified ROVER: vote at each position of the backbone
    # For each word in backbone, count votes from all hypotheses
    result_words = []

    for pos, backbone_word in enumerate(backbone):
        votes = {}  # word -> cumulative weight

        for hyp_idx, words in enumerate(word_lists):
            w = weights[hyp_idx]

            # Find best matching position in this hypothesis
            # Simple approach: check if backbone_word appears in hypothesis
            if pos < len(words):
                word = words[pos]
            else:
                word = "<NULL>"

            votes[word] = votes.get(word, 0) + w

        # Apply null penalty
        if "<NULL>" in votes:
            votes["<NULL>"] *= null_penalty

        # Select most-voted word
        best_word = max(votes, key=votes.get)
        if best_word != "<NULL>":
            result_words.append(best_word)

    return " ".join(result_words)
```

Note: The above is a simplified ROVER. A full implementation uses the NIST SCTK toolkit's `rover` command or a proper multiple-alignment algorithm (Fiscus, 1997).

Why This Is the Best Strategy for Lip Reading

ROVER operates at the word level, which is exactly right for our failure modes: - **Hallucinated words** appear in only 1-2 beams → get outvoted - **Correct words** tend to appear in many beams → get selected - **Word length** is implicitly controlled by the voting process

Literature

- Fiscus (1997). "A Post-Processing System to Yield Reduced Word Error Rates: Recognizer Output Voting Error Reduction (ROVER)." IEEE ASRU 1997.
- Evermann & Woodland (2000). "Posterior Probability Decoding, Confidence Estimation and System Combination." NIST Speech Transcription Workshop.

4.3 Strategy 3: N-Best Rescoring with External Model

Expected improvement: 10-20% relative WER reduction (but high complexity) **Complexity:** High

Concept

Use an external language model or scoring function to re-rank the N-best hypotheses. The beam search uses the model's internal LLM (LLaMA-2), but an external model may rank the hypotheses differently.

Rescoring Options

Rescorer	Description	Expected Benefit
GPT/Claude API	Send N hypotheses to an LLM, ask "which is most likely speech?"	High — powerful language model
Whisper (audio)	If audio is available, score hypotheses against audio ASR	Very high — but defeats visual-only purpose
N-gram LM	Fast, simple, domain-specific language model	Medium — catches unnatural phrases
BERT/ RoBERTa	Score hypotheses by masked language model likelihood	Medium — catches semantic errors

Implementation with External LLM

```
def rescore_with_llm(hypotheses, scores, topic_context=""):
    """Re-rank N-best using an external LLM."""

    prompt = f"""Given these {len(hypotheses)} possible lip-reading transcriptions
of a speaker {f'discussing {topic_context}' if topic_context else ''},
rank them from most to least likely to be correct speech:

{chr(10).join(f'{i+1}. "{h}" (score: {s:.2f})' for i, (h, s) in enumerate(zip(hypotheses, s

Return only the number of the most likely transcription."""
```

```
    # Call external LLM API
    response = call_llm_api(prompt)
    best_idx = parse_response(response)
    return hypotheses[best_idx]
```

Trade-off: This adds API cost and latency. Only viable for high-value content where accuracy justifies the cost.

4.4 Strategy 4: Confidence-Weighted Word Selection

Expected improvement: 3-8% relative WER reduction **Complexity:** Medium

Concept

Combine the N-best hypotheses at the word level, but instead of simple voting (ROVER), weight each word by its token-level confidence from the beam search scores.

```
def confidence_weighted_selection(hypotheses, token_confidences_per_hyp):
    """Select words based on per-token confidence across hypotheses.

    For each word position, select the word from the hypothesis
    where that position has the highest confidence.
    """
    # Align hypotheses (simplified: assume similar lengths)
    max_len = max(len(h.split()) for h in hypotheses)
    result = []

    for pos in range(max_len):
        best_word = None
        best_conf = -1

        for hyp_idx, hyp in enumerate(hypotheses):
            words = hyp.split()
            if pos < len(words):
                word = words[pos]
                conf = token_confidences_per_hyp[hyp_idx][pos]
                if conf > best_conf:
                    best_conf = conf
                    best_word = word

        if best_word:
            result.append(best_word)

    return " ".join(result)
```

This requires the token-level confidence from Report 4.

4.5 Strategy 5: N-Best Diversity Analysis (Meta-Confidence)

Expected improvement: Not a direct WER improvement, but enables quality estimation
Complexity: Low

Concept

Use the **diversity** among the N-best hypotheses as a confidence signal. If all 20 beams produce similar text, the model is confident. If the beams diverge wildly, the model is uncertain.

```
import editdistance

def beam_diversity_score(hypotheses):
    """Measure how diverse the N-best hypotheses are.

    Returns:
        diversity: float between 0 (all identical) and 1 (all different)
        agreement_words: words that appear in >50% of hypotheses
    """
    n = len(hypotheses)
    word_lists = [h.split() for h in hypotheses]

    # Pairwise WER between all hypotheses
    pairwise_wers = []
    for i in range(n):
        for j in range(i+1, n):
            dist = editdistance.eval(word_lists[i], word_lists[j])
            ref_len = max(len(word_lists[i]), len(word_lists[j]), 1)
            pairwise_wers.append(dist / ref_len)

    diversity = sum(pairwise_wers) / len(pairwise_wers) if pairwise_wers else 0

    # Find consensus words (appear in >50% of hypotheses)
    all_words = set()
    for wl in word_lists:
        all_words.update(wl)

    agreement_words = []
    for word in all_words:
        count = sum(1 for wl in word_lists if word in wl)
        if count > n * 0.5:
            agreement_words.append(word)

    return {
        'diversity': diversity,
        'agreement_words': agreement_words,
        'n_agreement': len(agreement_words)
    }
```

Why This Matters

This directly addresses the **separation problem** from Report 1. If beam diversity is low (all beams agree), the segment is likely well-transcribed. If diversity is high, the segment is likely ambiguous/hallucinated.

Expected correlation with WER: **r = 0.4-0.6** (stronger than any currently available signal).

---

5. Strategy Comparison

Strategy	Expected WER Reduction	Requires	Compute Cost	Complexity
MBR Decoding	5-10% relative	N-best + scores	Low (edit distance)	Medium
ROVER	5-15% relative	N-best + alignment	Medium (alignment)	Medium-High
External Rescoring	10-20% relative	N-best + API	High (LLM API)	High
Confidence-Weighted	3-8% relative	N-best + token confidence	Low	Medium
Diversity Analysis	Indirect (quality estimation)	N-best only	Low	Low

Recommendation: Implement **MBR** and **ROVER** first. They have the best improvement-to-complexity ratio and are well-established in the ASR literature. Add **Diversity Analysis** as a quality estimation signal (complementary to Report 4's confidence scoring).

---

6. Expected Impact on Current Results

6.1 Estimated WER Improvements

Applying the aggregation strategies to our english_1k results:

Metric	Current (Top-1)	MBR (est.)	ROVER (est.)	Combined Best (est.)
Mean WER	67.0%	~61-64%	~58-63%	~55-60%
Corpus WER	125.5%	~110-120%	~100-115%	~95-108%

Metric	Current (Top-1)	MBR (est.)	ROVER (est.)	Combined Best (est.)
Catastrophic (≥100%)	20.6%	~16-19%	~14-18%	~12-16%
Good (≤20%)	11.4%	~13-15%	~14-16%	~15-18%

These are conservative estimates based on ASR N-best combination literature, adjusted downward because: 1. Our beams are from the same model (less diversity than multi-system ROVER) 2. The domain gap limits how much aggregation can help 3. When the visual features are truly ambiguous, all beams may be equally wrong

6.2 Where Aggregation Helps Most

Failure Mode	Aggregation Benefit	Explanation
Insertions (extra words)	High	Extra words are unlikely to appear in multiple beams
Substitutions (wrong words)	Medium	Correct word may appear in some beams but not the top-1
Named entities	Low-Medium	If no beam captures the entity, aggregation can't help
Complete hallucination	Low	If all beams hallucinate (different things), aggregation still fails

6.3 Where Aggregation Doesn't Help

Aggregation cannot fix the **fundamental domain gap**. If the visual encoder fails to extract meaningful features (bad lighting, side profile, occluded mouth), all 20 beams will be equally wrong — they'll just be wrong in different ways. Aggregation helps when the correct answer is "in there somewhere" but isn't the top-1.

---

7. Full Implementation Roadmap

Phase 1: Capture N-Best (4-6 hours)

1. Modify `vsp_llm.py` generate() to accept `num_return_sequences`
2. Modify `vsp_llm_decode.py` to save N-best hypotheses and scores in JSON
3. Add `num_return_sequences` to decode YAML config
4. Test on 10 segments to verify correct output format

Output: JSON with `hypo_nbest` (list of N strings) and `scores_nbest` (list of N floats) per segment.

Phase 2: Implement MBR and ROVER (1-2 days)

1. Implement MBR decode function (pure Python, no dependencies)
2. Implement simplified ROVER (word-level voting)
3. Add as post-processing step in decode or report pipeline
4. Run on full english_1k dataset
5. Compare WER: top-1 vs. MBR vs. ROVER

Phase 3: Integrate with Reports (1 day)

1. Add aggregated hypothesis to report CSV
2. Show N-best alternatives in HTML report
3. Add beam diversity score to segment metadata
4. Color-code agreement words (appear in >50% of beams) vs. outlier words

Phase 4: Advanced Strategies (3-5 days)

1. Implement confidence-weighted word selection (requires Report 4)
2. Test external LLM rescoring on subset
3. Combine MBR + confidence + diversity into unified quality score

---

8. Interaction with Other Reports

This Report (5)	Interacts With	How
N-best extraction	Report 4 (Confidence)	Same code change enables both
Beam diversity	Report 1 (Quality estimation)	Diversity is a quality signal
MBR/ROVER	Report 2 (Hyperparameters)	Beam width affects N-best quality
External rescoring	Report 3 (Prompt engineering)	Context can improve rescoring

The recommended implementation order: 1. **Report 4 Phase 1** (sequence scores) — smallest change, biggest payoff 2. **Report 5 Phase 1** (N-best capture) — builds on same code change 3. **Report 2 experiments** (hyperparameter tuning) — independent, run in parallel 4. **Report 5 Phase 2** (MBR + ROVER) — uses N-best from Phase 1 5. **Report 3 experiments** (prompt engineering) — independent 6. **Report 4 Phase 2** (token-level confidence) — deeper integration

---

9. References

1. Fiscus (1997). "A Post-Processing System to Yield Reduced Word Error Rates: Recognizer Output Voting Error Reduction (ROVER)." IEEE ASRU 1997. — The foundational ROVER paper.
2. Kumar & Byrne (2004). "Minimum Bayes-Risk Decoding for Statistical Machine Translation." HLT-NAACL 2004. — MBR decoding theory.
3. Eikema & Aziz (2022). "Sampling-Based Approximations to Minimum Bayes Risk Decoding for Neural Machine Translation." EMNLP 2022. — Modern MBR for neural models.
4. Freitag et al. (2022). "High Quality Rather Than High Model Probability: MBR Decoding with Neural Metrics." TACL 2022. — MBR with learned metrics.
5. Sim et al. (2007). "Consensus Network Decoding for Statistical Machine Translation System Combination." IEEE ICASSP 2007. — Confusion network / consensus approaches.
6. Xu et al. (2011). "Minimum Bayes Risk Decoding and System Combination Based on a Recursion for Edit Distance." Computer Speech & Language 25(4). — Efficient MBR computation.
7. Evermann & Woodland (2000). "Posterior Probability Decoding, Confidence Estimation and System Combination." NIST Speech Transcription Workshop. — Score-based combination.
8. Mangu et al. (2000). "Finding Consensus in Speech Recognition: Word Error Minimization and Other Applications of Confusion Networks." Computer Speech and Language 14(4). — Confusion network theory.
9. Yeo et al. (2024). "VSP-LLM." arXiv:2402.15151. — The VSP-LLM model paper.
10. HuggingFace Transformers. "Generation Strategies — num_return_sequences." — API documentation.